

3-1 資料流程觀念：抓取 → 清洗 → 儲存 → 分析

一、觀念介紹：資料的「採礦與精煉」

取得資料只是第一步，完整流程包含四階段：

1. 抓取 (Crawl)

- 使用 requests / Selenium 取得資料
- 原始資料雜亂（如礦石）

2. 清洗 (Clean) — CH3 核心

- 去重、補值、格式轉換
- 轉為結構化表格

3. 儲存 (Store)

- 存成 CSV / Excel / 資料庫
- 可重複使用

4. 分析 (Analyze)

- 計算、視覺化
- 產出洞察

二、程式範例：體驗「一條龍」處理

範例 1：電商價格快速清洗

假設抓回來的價格帶有「\$」和「,」，且有重複資料。

```
import pandas as pd
```

```
# 模擬抓取回來的原始資料 (可能有重複，格式混亂)
```

```
raw_data = [
    {"item": "iPhone 15", "price": "$29,900"},
    {"item": "iPhone 15", "price": "$29,900"},
    {"item": "iPad Air", "price": "$19,500"},
    {"item": "MacBook", "price": "None"} # 缺失值
]
```

```
# 轉成 Pandas DataFrame (表格)
```

```
df = pd.DataFrame(raw_data)
```

```
# 執行清洗：去重、移除符號、轉數字
```

```
df = df.drop_duplicates() # 1. 去除重複
```

```
df['price'] = df['price'].str.replace('$', '').str.replace(',', '') # 2. 取代符號
```

```
df['price'] = pd.to_numeric(df['price'], errors='coerce') # 3. 轉為數字，轉不動的變空值
```

```
print("清洗後的乾淨資料：")
```

```
print(df)
```

範例 2：政府 Open Data 的快速預覽

Pandas 讀取 CSV 的速度與自動解析能力遠超乎想像。

```
import pandas as pd
# 模擬從網路讀取一個天氣 CSV (此處用讀取檔案代替)
df = pd.read_csv("https://opendata.example.com/weather.csv")
# 假設讀取後長這樣
df = pd.DataFrame({
    "城市": ["台北", "台中", "高雄"],
    "溫度": [25, 28, 30]
})
# 快速分析：只要一行
print(f'平均溫度：{df["溫度"].mean()} 度')
```

範例 3：多來源資料合併 (分析前奏)

爬蟲常分批抓取，Pandas 能像 Excel 樞紐分析一樣快速整合。

```
import pandas as pd
# 批次一：早上的新聞
df1 = pd.DataFrame({"title": ["新聞 A"], "clicks": [100]})
# 批次二：下午的新聞
df2 = pd.DataFrame({"title": ["新聞 B"], "clicks": [250]})
# 合併
all_news = pd.concat([df1, df2], ignore_index=True)
print(all_news)
```

三、程式說明

- import pandas as pd：標準寫法
- DataFrame：類似 Excel 表格
- drop_duplicates()：去除重複資料
- str.replace()：整欄字串處理
- pd.to_numeric()：轉數值

四、常見錯誤

1. 未清洗直接分析 → 字串無法計算
2. 索引錯亂 → 使用 ignore_index=True
3. 過度依賴 Excel → 大數據應使用 Pandas

五、延伸重要觀念

- ETL：Extract → Transform → Load
- 冪等性：多次執行結果一致

補充範例：讓學生更有感的 Pandas 威力

範例 4：快速過濾「高 CP 值」商品 (3-3 預演)

```
import pandas as pd
products = [
    {"name": "咖啡機", "price": 5000, "rating": 4.8},
    {"name": "磨豆機", "price": 1200, "rating": 4.2},
    {"name": "濾紙", "price": 150, "rating": 4.9},
    {"name": "保溫杯", "price": 800, "rating": 3.5}
]
df = pd.DataFrame(products)
```

只要一行：篩選分數大於 4.5 且價格低於 2000 的商品

```
best_buy = df[(df['rating'] > 4.5) & (df['price'] < 2000)]
print("推薦購買：")
print(best_buy)
```

範例 5：處理「多餘空白」與「格式統一」 (3-4 預演)

爬蟲抓下來的文字常帶有 \n 或空格，Pandas 處理起來極快。

```
import pandas as pd

df = pd.DataFrame({
    "user": [" Alice ", "Bob\n", "  Charlie  "],
    "email": ["ALICE@gmail.com", "bob@Gmail.com", "CHARLIE@outlook.com"]
})
```

清除空格並統一轉小寫

```
df['user'] = df['user'].str.strip()
df['email'] = df['email'].str.lower()
print(df)
```

3-1 隨堂練習

一、選擇題 (5 題)

1. () 在資料處理的四個階段中，專業數據專案最耗時、也是 Pandas 發揮核心作用的是哪一階段？(A) 抓取 (Crawl) (B) 清洗 (Clean) (C) 儲存 (Store) (D) 分析 (Analyze)。
2. () 業界常說的「ETL」流程中，字母 T 代表 Transform (轉換)，這對應到我們課堂講的哪個步驟？(A) 抓取 (B) 清洗 (C) 儲存 (D) 分析。
3. () 當爬蟲抓回來的價格資料帶有「\$」符號時 (例如 "\$1,200")，若想進行平均值計算，第一步該怎麼做？(A) 直接計算 (B) 用 `pd.to_numeric()` 強制轉型 (C) 先用 `str.replace()` 去除符號 (D) 存入 Excel 手動刪除。
4. () 如果在合併兩份爬蟲資料後，發現索引 (Index) 出現重複 (如 0, 1, 0, 1)，應在合併時加入哪個參數？(A) `ignore_index=True` (B) `drop=True` (C) `clean=True` (D) `fix_index=True`。
5. () 關於「垃圾進，垃圾出 (GIGO)」的觀念，以下敘述何者正確？(A) 只要程式碼沒噴錯，資料就是對的 (B) 只要抓取的數量夠多，就不需要清洗 (C) 沒清洗的資料會導致錯誤的分析結果 (D) Pandas 會自動幫你修正所有錯誤格式。

二、填充題 (5 題)

1. 在 Pandas 中，若要刪除 DataFrame 中完全重複的資料行，可以使用 `df._____()` 方法。
2. 为了方便呼叫，業界慣例會將 Pandas 縮寫為 _____。
3. 爬蟲抓回來的資料通常是字串，若要一次性處理整欄字串 (如去空格、轉大小寫)，通常會先加上 _____ 屬性再串接方法。
4. 資料處理流程中，將清洗好的資料存入 CSV 或資料庫的動作稱為 _____。
5. 一個專業流程應確保執行一次與執行十次的結果相同，這種特性稱為「_____ 性」。

三、實作題 (2 題)

任務一：清理髒資料 請建立一個包含兩筆資料的 DataFrame，欄位為 `product` 與 `price`。資料內容為：`{"product": [" iPhone ", "iPad"], "price": ["$30,000", "$20,000"]}`。請完成以下清洗動作：

- (1) 去除 `product` 欄位多餘的空格。
- (2) 去除 `price` 欄位的 \$ 與 , 並轉為數值型態。

任務二：批次資料合併 現有兩份爬蟲抓回來的數據：`df_a = pd.DataFrame({"news": ["頭條 1"], "views": [500]})` `df_b = pd.DataFrame({"news": ["頭條 2"], "views": [300]})` 請將兩份資料合併為 `df_all`，確保索引不重複，並印出合併後的「總觀看數 (views)」。

3-2 DataFrame 與 Series：把資料想像成 Excel 表格

一、觀念介紹：Pandas 的兩大支柱

- Series (一維) → 類似一欄或一行
- DataFrame (二維) → 類似整張表格 (由多個 Series 組成)

二、程式範例

範例 1：手動建立 DataFrame (來自字典)

這是最直觀的建立方式，Key 變成欄位名，Value (List) 變成資料。

```
import pandas as pd
```

```
# 準備資料
```

```
data = {  
    "日期": ["2026-04-13", "2026-04-14", "2026-04-15"],  
    "收盤價": [810, 815, 805],  
    "漲跌": [5, 5, -10]  
}
```

```
# 建立 DataFrame
```

```
df = pd.DataFrame(data)
```

```
print("--- 股票小表格 ---")
```

```
print(df)
```

```
# 觀察其中一欄 (這就是 Series)
```

```
print("\n--- 僅觀察收盤價欄位 (Series) ---")
```

```
print(df["收盤價"])
```

範例 2：查看資料的「體質」 (屬性檢查)

拿到資料後，第一步不是分析，而是先確認資料長什麼樣子。

```
# 延續上面的 df
```

```
print(f"資料形狀 (列, 欄): {df.shape}")
```

```
print(f"欄位名稱: {df.columns}")
```

```
print(f"資料型態: \n{df.dtypes}") # 檢查數字是否真的被當成數字處理
```

範例 3：快速預覽大型資料

當你抓回一萬筆資料，不可能全部印出來看，這兩個指令是救星。

```
# 假設 df 有很多列
```

```
print(df.head(2)) # 只看前 2 列
```

```
print(df.tail(1)) # 只看最後 1 列
```

```
print(df.describe()) # 快速計算：平均值、標準差、最大最小值 (數據分析神招)
```

範例 4：Series 的索引特性

Series 就像是有名字的 List。

```
s = pd.Series([100, 200, 300], index=["台北", "台中", "高雄"])
print(s)
print(f'台中的數值是: {s["台中"]}')
```

三、程式說明

- df.shape：資料大小（列，欄）
- df.columns：欄位名稱
- df.dtypes：資料型態（非常重要）
- df.describe()：快速統計分析
- df["欄位"]：取出單一欄（Series）

四、常見錯誤

1. 欄位名稱錯誤 → 會出現 KeyError
2. 誤當 List 使用 → DataFrame 需用 loc / iloc
3. 忘記 import → 無法使用 pd

五、延伸重要觀念

- 向量化運算
→ 避免 for 迴圈（如 df['salary'] * 1.1）
- 維度概念
→ Series：一維
→ DataFrame：二維

3-2 隨堂練習

一、選擇題 (5 題)

1. () Pandas 中，代表「整張二維表格」的物件名稱是？ (A) Series (B) DataFrame (C) List (D) Dictionary。
2. () 如果想看 DataFrame 前 5 筆資料，應該使用哪個方法？ (A) df.first() (B) df.show() (C) df.head() (D) df.tail()。
3. () 下列哪一個屬性可以告訴你資料表的「列數」與「欄數」？ (A) df.columns (B) df.shape (C) df.size (D) df.info。
4. () Series 與 List 最主要的差別在於 Series 具有什麼？ (A) 括號 (B) 索引 (Index) (C) 只有數字 (D) 執行速度。
5. () 執行 df.describe() 時，它主要會針對哪種型態的欄位進行運算？ (A) 字串 (B) 日期 (C) 數值 (D) 布林值。

二、填充題 (5 題)

1. 在 Pandas 中，一維的資料結構稱為 _____。
2. 若要檢查 DataFrame 中每一欄的資料型態 (例如是 int 還是 object)，要查看 df._____。
3. 想要查看資料表的最後三筆資料，應輸入 df._____。
4. 將字典轉成 DataFrame 時，字典的 Key 會自動變成表格的 _____ 名稱。
5. DataFrame 物件是由多個 _____ 拼接而成的。

三、實作題 (2 題)

任務一：建立員工清單 請建立一個 DataFrame 名為 staff_df，包含三個欄位：姓名、部門、薪資。隨意填入 3 位員工資料後，印出該資料表的「欄位名稱」與「形狀」。

任務二：Series 運算 建立一個 Series，索引為 ["週一", "週二", "週三"]，數值為 [120, 150, 90] (代表外送訂單數)。請計算並印出這三天的「總訂單數」與「平均訂單數」。

3-3 資料篩選與過濾：loc、iloc 與條件判斷

在學 loc / iloc 前，先看三種常見但容易踩雷的寫法。它們好用，但容易造成混亂。

一、常見過濾方式與其「副作用」

1. 直接索引 (Square Brackets)

這是最直覺的方式。

範例 1：選取單一欄位

取得「姓名」這一欄

```
names = df['姓名']
```

範例 2：選取多個欄位

取得「姓名」與「薪資」

```
subset = df[['姓名', '薪資']]
```

- 遭遇的問題：語法歧義

df['姓名'] 是「欄」，但 df[0:3] 是「列」。當需要同時選列與欄時會變得混亂且難維護。

2. 布林遮罩 (Boolean Masking)

用條件判斷篩選資料。

範例 1：單一條件

找出薪資大於 50,000 的人

```
high_pay = df[df['薪資'] > 50000]
```

範例 2：多重條件

找出「業務部」且「薪資 > 40000」的人

```
target = df[(df['部門'] == '業務') & (df['薪資'] > 40000)]
```

- 遭遇的問題：可讀性差、括號多，且修改資料時容易出現 SettingWithCopyWarning (不確定是否改到原資料)。

3. query() 方法

用類似 SQL 的語法查詢。

範例 1：簡單查詢

```
result = df.query('薪資 > 45000')
```

範例 2：使用外部變數

```
min_pay = 40000
```

```
result = df.query('薪資 > @min_pay')
```

- 遭遇的問題：彈性不足。邏輯複雜時 (如自訂函數) 不易維護與除錯。

二、為什麼我們最終需要 loc 與 iloc ?

為了解決歧義問題，Pandas 提供標準定位方式：統一使用 [列, 欄]。

核心概念：「標籤 (loc)」 vs 「位置 (iloc)」

1. iloc：位置索引，只看位置 (從 0 開始)。

範例 1：

```
data = df.iloc[0, 1]
```

範例 2：

```
subset = df.iloc[0:3, 0:2]
```

2. loc：標籤索引，用欄位名稱與索引名稱。

範例 1：

```
salary = df.loc['A01', '薪資']
```

範例 2：

```
report = df.loc[df['部門'] == '業務', ['姓名', '薪資']]
```

3. 隨堂觀念檢查

需求場景	推薦工具	理由
看前 10 筆	iloc	位置固定
找缺貨商品	loc + 條件	直觀且可選欄
修改資料	loc	安全、不會警告

三、延伸重要觀念

- 記憶法：
 - iloc → Integer (用數字)
 - loc → Label (用名稱)
- 布林篩選本質：
 - df[df['age'] > 20] 是先產生 True/False，再取出 True 的資料。

3-3 隨堂練習

一、選擇題 (5 題)

1. () 如果想使用「整數編號」來選取 DataFrame 的資料，應該使用哪個方法？ (A) loc (B) iloc (C) query (D) filter。
2. () `df.loc[df['價格'] > 100, '品名']` 這行程式碼中，逗號後面的 '品名' 代表什麼？ (A) 要篩選的條件 (B) 要顯示的欄位 (C) 要刪除的資料 (D) 新增的索引。
3. () 在 `iloc[0:3, 1:4]` 中，選取的「列」範圍是？ (A) 第 0, 1, 2, 3 列 (B) 第 1, 2, 3 列 (C) 第 0, 1, 2 列 (D) 只有第 0 列。
4. () 為什麼在修改資料時，使用 loc 會比直接索引安全？ (A) loc 速度比較快 (B) loc 不佔記憶體 (C) loc 能確保直接操作在原始 DataFrame 上，避免產生警告 (D) loc 是 Python 的保留字。
5. () `df.iloc[:, 0]` 代表什麼意思？ (A) 第一列的所有資料 (B) 第一欄的所有資料 (C) 整張表格 (D) 報錯。

二、填充題 (5 題)

1. loc 定位是基於標籤名稱，而 iloc 定位是基於 _____。
2. 在 `df.loc[條件, 欄位]` 結構中，若想選取「所有欄位」，逗號後方應填入 _____ (符號)。
3. 使用多重條件過濾時，若希望兩個條件同時成立，應使用 _____ 符號 (例如：且)。
4. 若要取得 DataFrame 的第 5 列到最後一列，iloc 的切片寫法為 `df.iloc[____; :]`。
5. 使用 `.query()` 方法時，若要在字串中使用外部變數，該變數前方需加上 _____ 符號。

三、實作題 (2 題)

```
raw_data = {  
    "產品": ["拿鐵", "卡布奇諾", "美式咖啡", "摩卡", "焦糖瑪奇朵"],  
    "價格": [120, 130, 100, 150, 160],  
    "庫存": [50, 30, 80, 20, 10]  
}
```

任務一：精準定位修改 承接上面的咖啡店範例，請用 **loc** 將「美式咖啡」的「庫存」修改為 120，並印出修改後的結果。

任務二：複雜篩選 請寫出一段程式碼，篩選出「價格介於 100 到 150 之間」且「庫存低於 40」的所有產品。

3-4 髒資料處理：去除重複值 (drop_duplicates) 與缺失值 (fillna)

一、觀念介紹：什麼是髒資料？

- 重複值 (Duplicates)：同一筆資料出現多次
- 缺失值 (NaN)：資料缺漏，Pandas 以 NaN 表示

二、完整實戰範例：連鎖店銷售記錄清洗

```
import pandas as pd
import numpy as np # 用來產生 NaN 空值
data = {
    "訂單編號": ["A01", "A02", "A01", "A03", "A04", "A05"],
    "品名": ["美式咖啡", "拿鐵咖啡", "美式咖啡", "卡布奇諾", None, "焦糖瑪奇朵"],
    "數量": [2, 1, 2, np.nan, 1, 3],
    "金額": [160, 120, 160, 120, 150, np.nan]
}
df = pd.DataFrame(data)
print("--- 原始髒資料 ---")
print(df)
```

第一步：處理重複值 (drop_duplicates)

```
print(f"\n 是否有重複資料：\n{df.duplicated()}")
df_clean = df.drop_duplicates()
print("\n--- 處理完重複值後的資料 ---")
print(df_clean)
```

第二步：處理缺失值 (fillna / dropna)

```
df_clean['品名'] = df_clean['品名'].fillna("未分類")
df_clean['數量'] = df_clean['數量'].fillna(0)
avg_price = df_clean['金額'].mean()
df_clean['金額'] = df_clean['金額'].fillna(avg_price)
print("\n--- 最終清洗完成的資料 ---")
print(df_clean)
```

三、關鍵指令對照

指令	用途	常用參數
df.duplicated()	找重複	subset=['欄位']
df.drop_duplicates()	刪重複	inplace=True
df.isnull()	找空值	.sum()
df.fillna(X)	補值	可用 dict
df.dropna()	刪空值	how='all'

四、延伸重要觀念

- **inplace** 建議少用：
建議寫 `df = df.dropna()`，避免警告且可讀性更好
- **補值策略**：
不一定補 0，可用平均值或前值 (`method='ffill'`)，依情境決定

3-4 隨堂練習

一、選擇題 (5 題)

1. () 在 Pandas 中，空值通常會顯示為什麼字樣？ (A) Null (B) Empty (C) NaN (D) Zero。
2. () 如果想找出 DataFrame 中「各欄位分別有多少個空值」，最快的方式是？ (A) df.count() (B) df.isnull().sum() (C) df.describe() (D) df.show_empty()。
3. () df.drop_duplicates(subset=['手機號碼']) 的作用是？ (A) 刪除整張表的重複項 (B) 只要手機號碼重複，就視為重複並刪除 (C) 刪除手機號碼欄位 (D) 搜尋手機號碼。
4. () 如果想將薪資欄位的空值全部補上「0」，應該使用？ (A) fillna(0) (B) dropna(0) (C) replace(0) (D) fix(0)。
5. () 執行 df.dropna() 後，發生了什麼事？ (A) 所有的 0 都消失了 (B) 只有包含空值的格子被清空 (C) 只要該列包含任何空值，整列都會被移除 (D) 程式會報錯。

二、填充題 (5 題)

1. 刪除重複值的指令是 drop_____。
2. 填補空值的指令是 _____。
3. 若要檢查某一行資料是否與之前的資料完全相同，可使用 df._____()。
4. 在清洗資料時，若想直接修改原始 DataFrame 而不產生新變數，通常需設定參數 inplace=_____。
5. 使用 dropna() 時，若希望「整列都是空值」才刪除，需設定參數 how='_____」。

三、實作題 (2 題)

任務一：重複資料清理

有一份學生簽到表 `names = ["小明", "小華", "小明", "小美", "小華"]`。請將其轉為 DataFrame 後，刪除重複的名字，並計算最後剩下幾個人。

任務二：缺失業績補強

某業務員的月營收為 `[30000, 45000, np.nan, 38000]`。請將這筆資料建立為 Series，並用該業務員有紀錄的月份平均值來填補那個缺失的月份。

3-5 時間序列：將字串轉為時間格式，進行年/月/日統計分析

一、觀念介紹：.dt 存取器的威力

當欄位轉為時間格式後，可使用 .dt 工具快速操作：

- 提取：年、月、日、星期、時
- 計算：日期差距
- 分組：依月、季統計

二、完整實戰範例：咖啡店營收時間分析

我們準備一份跨月份、包含時間字串的銷售資料。

```
import pandas as pd
```

```
data = {  
    "交易時間": ["2026/01/05 10:30", "2026/01/20 14:00", "2026/02/10 09:15", "2026/02/15  
16:45", "2026/03/01 11:00"],  
    "金額": [150, 200, 120, 350, 400]  
}
```

```
df = pd.DataFrame(data)  
print("--- 原始資料 (交易時間目前是字串) ---")  
print(df.dtypes)
```

第一步：轉換為時間格式 (pd.to_datetime)

```
# 轉換格式
```

```
df["交易時間"] = pd.to_datetime(df["交易時間"])  
print("\n--- 轉換後 ---")  
print(df.dtypes) # 型態會變成 datetime64[ns]
```

第二步：提取時間資訊 (年/月/日/星期)

```
# 使用 .dt 存取器
```

```
df["年份"] = df["交易時間"].dt.year  
df["月份"] = df["交易時間"].dt.month  
df["日"] = df["交易時間"].dt.day  
df["星期幾"] = df["交易時間"].dt.day_name() # 顯示英文星期名稱  
print("\n--- 提取時間資訊後 ---")  
print(df[["交易時間", "月份", "星期幾", "金額"]])
```

第三步：時間統計分析 (按月加總)

```
# 依照月份進行分組加總
monthly_revenue = df.groupby("月份")["金額"].sum()
print("\n--- 每月營收統計 ---")
print(monthly_revenue)
```

第四步：時間篩選 (只看三月份的資料)

```
# 篩選 2026 年 3 月之後的訂單
march_data = df[df["交易時間"] >= "2026-03-01"]
print("\n--- 三月訂單 ---")
print(march_data)
```

三、關鍵指令小工具

指令屬性	說明	範例結果
.dt.year	年份	2026
.dt.month	月份	4
.dt.weekday	星期 (0=週一)	0
.dt.hour	小時	14
.dt.strftime('%Y/%m')	格式化輸出	"2026/04"

四、延伸重要觀念

- dt 存取器：轉成 datetime64 後即可使用時間功能
- 重取樣 (Resampling)：可將高頻資料 (分鐘) 轉為低頻 (小時、日)

3-5 隨堂練習

一、選擇題 (5 題)

1. () 在 Pandas 中，將字串轉換為時間格式的函數名稱是？ (A) `pd.is_time()` (B) `pd.to_datetime()` (C) `pd.convert_date()` (D) `pd.str_to_time()`。
2. () 轉換成時間格式後，我們可以使用哪一個存取器來提取「月份」或「年份」資訊？ (A) `.str` (B) `.map` (C) `.dt` (D) `.time`。
3. () `df['date'].dt.weekday` 回傳的數值中，「0」通常代表星期幾？ (A) 星期日 (B) 星期六 (C) 星期二 (D) 星期一。
4. () 如果想將時間格式化為「2026-04」這種只有年月的方式，應使用？ (A) `.dt.strftime()` (B) `.dt.format()` (C) `.dt.string()` (D) `.dt.to_month()`。
5. () 為什麼需要將日期字串轉為 Datetime 格式？ (A) 檔案會變小 (B) 才能進行時間排序、分組運算與篩選 (C) 這是 Python 的強制要求 (D) 為了讓顯示顏色變漂亮。

二、填充題 (5 題)

1. 在 Pandas 時間序列中，提取小時資訊的屬性是 `.dt._____`。
2. 若要計算兩個日期欄位相減後的天數，結果的資料型態會稱為 `Time_____`。
3. `pd.to_datetime()` 若遇到無法解析的錯誤字串，可設定參數 `errors='_____'` 將其轉為空值。
4. 想要取得日期的英文星期名稱（如 Monday），可使用 `.dt._____()` 方法。
5. 進行時間分組統計時，通常會搭配 `df._____("月份")["金額"].sum()` 這種組合技。

三、實作題 (2 題)

任務一：假日流量分析 有一組網站造訪日期 `["2026-04-11", "2026-04-12", "2026-04-13"]`（分別是週六、週日、週一）。請將其轉為時間格式，並多增加一欄名為 `is_weekend`，內容為該日期是否為星期六或星期日（提示：可以使用 `.dt.weekday` 是否大於等於 5 來判斷）。

任務二：早鳥優惠篩選 有一組外送訂單時間 `["08:30", "11:00", "07:15", "13:00"]`。請轉為時間格式後，篩選出所有「早上 9 點以前」的訂單。

3-5 隨堂練習

一、選擇題 (5 題)

6. () 在 Pandas 中，將字串轉換為時間格式的函數名稱是？ (A) `pd.is_time()` (B) `pd.to_datetime()` (C) `pd.convert_date()` (D) `pd.str_to_time()`。
7. () 轉換成時間格式後，我們可以使用哪一個存取器來提取「月份」或「年份」資訊？ (A) `.str` (B) `.map` (C) `.dt` (D) `.time`。
8. () `df['date'].dt.weekday` 回傳的數值中，「0」通常代表星期幾？ (A) 星期日 (B) 星期六 (C) 星期二 (D) 星期一。
9. () 如果想將時間格式化為「2026-04」這種只有年月的方式，應使用？ (A) `.dt.strftime()` (B) `.dt.format()` (C) `.dt.string()` (D) `.dt.to_month()`。
10. () 為什麼需要將日期字串轉為 Datetime 格式？ (A) 檔案會變小 (B) 才能進行時間排序、分組運算與篩選 (C) 這是 Python 的強制要求 (D) 為了讓顯示顏色變漂亮。

二、填充題 (5 題)

6. 在 Pandas 時間序列中，提取小時資訊的屬性是 `.dt._____`。
7. 若要計算兩個日期欄位相減後的天數，結果的資料型態會稱為 `Time_____`。
8. `pd.to_datetime()` 若遇到無法解析的錯誤字串，可設定參數 `errors='_____'` 將其轉為空值。
9. 想要取得日期的英文星期名稱（如 Monday），可使用 `.dt._____()` 方法。
10. 進行時間分組統計時，通常會搭配 `df._____("月份")["金額"].sum()` 這種組合技。

三、實作題 (2 題)

任務一：假日流量分析 有一組網站造訪日期 `["2026-04-11", "2026-04-12", "2026-04-13"]`（分別是週六、週日、週一）。請將其轉為時間格式，並多增加一欄名為 `is_weekend`，內容為該日期是否為星期六或星期日（提示：可以使用 `.dt.weekday` 是否大於等於 5 來判斷）。

任務二：早鳥優惠篩選 有一組外送訂單時間 `["08:30", "11:00", "07:15", "13:00"]`。請轉為時間格式後，篩選出所有「早上 9 點以前」的訂單。

3-6 資料匯出與視覺化入門：讓數據說話

一、觀念介紹：從工程到決策

資料處理的最終目標：

- 結構化：轉為 Excel/CSV
- 摘要化：只看關鍵指標（總和、成長）
- 視覺化：快速看出趨勢與異常

二、完整實戰範例

範例 1：多維度摘要統計 (groupby + agg)

```
import pandas as pd
```

```
# 模擬連鎖店銷售明細
```

```
data = {  
    "區域": ["北區", "北區", "中區", "南區", "北區", "中區"],  
    "店名": ["台北店", "新北店", "台中店", "高雄店", "台北店", "台中店"],  
    "營收": [5000, 3000, 4500, 6000, 5200, 4800],  
    "客數": [50, 40, 45, 70, 55, 50]  
}
```

```
df = pd.DataFrame(data)
```

```
# 任務：計算每個「區域」的「營收總和」與「平均每客單價」
```

```
summary = df.groupby("區域").agg({  
    "營收": ["sum", "mean"],  
    "客數": "sum"  
})
```

```
print("--- 區域營收摘要 ---")
```

```
print(summary)
```

範例 2：匯出整潔報表 (CSV 與 Excel)

```
# 匯出成 CSV
```

```
df.to_csv("daily_report.csv", index=False, encoding="utf-8-sig")
```

```
# 匯出成 Excel
```

```
df.to_excel("daily_report.xlsx", index=False, sheet_name="4 月銷售")
```

```
print("報表匯出成功！")
```

範例 3：Matplotlib 一行畫出趨勢圖

```
import matplotlib.pyplot as plt

# 解決 Matplotlib 中文顯示問題
plt.rcParams['font.sans-serif'] = ['Microsoft JhengHei']

# 模擬一週營收趨勢
trend_data = {
    "日期": pd.to_datetime(["2026-04-06", "2026-04-07", "2026-04-08", "2026-04-09",
                             "2026-04-10"]),
    "業績": [12000, 15000, 11000, 18000, 22000]
}
df_trend = pd.DataFrame(trend_data)

# 畫圖
df_trend.plot(x="日期", y="業績", kind="line", marker='o', title="本週業績趨勢圖")
plt.ylabel("新台幣")
plt.grid(True)
plt.show()
```

三、讓主管看懂的資料呈現重點

- 使用清楚的欄位名稱（避免原始資料格式）
- 圖表需標示異常原因
- CSV 匯出需用 utf-8-sig 避免亂碼

四、延伸重要觀念

- EDA（探索性資料分析）：用圖快速檢查資料品質（如異常值）
- Matplotlib 基礎：Pandas .plot() 底層即為 Matplotlib，進階設定需直接操作它

3-6 隨堂練習

一、選擇題 (5 題)

1. () 如果想把 DataFrame 儲存為 Excel 檔案，應該使用哪個方法？ (A) to_csv() (B) save_excel() (C) to_excel() (D) export_xlsx()。
2. () 在 to_csv 時，為了避免 Excel 開啟中文時出現亂碼，編碼應設為？ (A) utf-8 (B) utf-8-sig (C) ascii (D) big5。
3. () groupby().agg() 的主要用途是什麼？ (A) 刪除重複資料 (B) 對資料進行分組並同時執行多種統計運算 (C) 改變資料的排列順序 (D) 幫資料畫圖。
4. () 使用 Matplotlib 畫圖時，如果想畫出「長條圖」，kind 參數應設為？ (A) line (B) pie (C) bar (D) scatter。
5. () 在 to_excel 方法中，參數 index=False 的作用是？ (A) 不隱藏欄位 (B) 不匯出最左側的索引編號 (C) 不儲存公式 (D) 不建立分頁。

二、填充題 (5 題)

1. 若要使用 Python 畫出基礎圖表，最常引用的庫是 import _____ as plt。
2. 在 Pandas 中，df.to_csv("data.csv", index=_____) 可以讓匯出的檔案不包含列索引。
3. 若想在 agg 中同時算出營收的總和與平均，寫法為 agg({"營收": ["sum", "_____"]})。
4. Matplotlib 若要顯示中文字體不產生亂碼，通常需要修改 plt.rcParams['_____']。
5. df.plot(kind='_____') 會產生圓餅圖。

三、實作題 (2 題)

任務一：貿易商報表匯出 建立一個包含 產品、單價、出口國 的 DataFrame，並將其匯成一個名為 export_list.csv 的檔案，要求中文不亂碼且不含索引。

任務二：餐飲業分店摘要 有一份各店業績表 df 包含 分店、日期、營收。請寫出程式碼：先按「分店」分組，算出每家店的「最高業績」與「最低業績」，最後將此統計結果畫成一張「長條圖 (bar)」。

CH3 課後綜合練習：Pandas 數據清洗

一、選擇題 (共 15 題)

- () 在 Pandas 中，下列哪一個指令可以快速查看數值欄位的平均值、最大值與四分位數？ (A) df.info() (B) df.describe() (C) df.values() (D) df.sum()。
- () 若要取得 DataFrame 的前 10 筆資料，應使用？ (A) df.head(10) (B) df.tail(10) (C) df.top(10) (D) df.first(10)。
- () 下列關於 loc 與 iloc 的敘述，何者正確？ (A) loc 用於數字位置，iloc 用於標籤名稱 (B) 兩者功能完全相同 (C) loc 用於標籤名稱，iloc 用於數字位置 (D) iloc 無法進行切片。
- () 使用 df.drop_duplicates() 時，若想針對「特定的某一欄」檢查重複，應設定哪個參數？ (A) inplace (B) columns (C) subset (D) by。
- () 在 Pandas 中，表示「缺失值 (空值)」的符號通常是？ (A) None (B) Null (C) NaN (D) Empty。
- () 執行 df.fillna(0) 的主要目的是？ (A) 刪除所有為 0 的資料 (B) 將所有空值補上數字 0 (C) 將所有資料清空 (D) 重新設定索引為 0。
- () 若要將字串格式的日期轉換為 Pandas 可運算的時間格式，應使用？ (A) pd.to_datetime() (B) pd.is_time() (C) pd.set_date() (D) pd.as_type('time')。
- () 透過 .dt 存取器取得「星期幾」時，回傳值為 0 代表？ (A) 星期日 (B) 星期一 (C) 星期五 (D) 星期六。
- () df.loc[df['價格'] > 500, '品名'] 代表什麼意義？ (A) 找出價格 > 500 的整張表 (B) 修改價格 > 500 的品名為 0 (C) 選取價格 > 500 的「品名」欄位資料 (D) 統計價格 > 500 的總數。
- () 當你想將 DataFrame 匯出成 Excel 檔案時，應使用哪個方法？ (A) to_csv() (B) to_excel() (C) save_xlsx() (D) export_file()。
- () 執行 df.dropna(how='all') 會發生什麼事？ (A) 只要有空值就刪除該列 (B) 只有當整列都是空值時才刪除 (C) 刪除所有包含數字的列 (D) 刪除標題列。
- () 下列哪一個符號在 Pandas 多重條件篩選中代表「或 (OR)」？ (A) & (B) ! (C) | (D) @。
- () 若要對「類別」欄位進行分組並計算「金額」總和，正確寫法為？ (A) df.group('類別').sum('金額') (B) df.groupby('類別')['金額'].sum() (C) df.sum().by('類別') (D) df.agg('類別', 'sum')。
- () 使用 to_csv() 時，設定 index=False 的作用是？ (A) 不隱藏標題 (B) 不儲存最左側的流水號索引 (C) 讓檔案變唯讀 (D) 不儲存數值。
- () Matplotlib 畫圖時，若要將圖表標題設定為「銷售分析」，應使用？ (A) plt.name() (B) plt.label() (C) plt.title() (D) plt.header()。

二、填充題 (共 10 題)

1. Pandas 的二維表格物件稱為 _____，一維則稱為 Series。
2. 想要知道 DataFrame 有幾列幾欄，可以查看 df._____ 屬性。
3. 篩選「價格在 100 到 200 之間」的條件寫法為：df[(df['價格'] >= 100) ____ (df['價格'] <= 200)]。
4. 匯出 CSV 檔案時，為了解決 Excel 開啟中文亂碼，建議設定 encoding='_____ '。
5. 時間序列中，提取「月份」資訊的寫法是 df['日期'].dt._____。
6. df.iloc[0:5, :] 會選取前 _____ 筆資料 (填數字)。
7. 若要刪除包含空值的整列資料，應使用 df._____() 方法。
8. 修改原始 DataFrame 而不產生新變數，通常需設定 inplace=_____。
9. 分組統計時，若想一次算出總和、平均與最大值，可使用 ._____() 方法。
10. pd.read_csv() 預設使用的欄位分隔符號是 _____。

三、實作題 (共 2 題)

任務一：電商資料清洗

現有一份髒資料 raw_df，請撰寫程式執行以下步驟：

1. 刪除「訂單編號」重複的列。
2. 將「金額」欄位的空值填補為該欄位的**平均值**。
3. 篩選出「金額 > 1000」且「狀態為 '已出貨'」的資料。

```
import pandas as pd
import numpy as np
data = { "訂單編號": ["1", "2", "1", "3", "4"],
         "金額": [1200, 800, 1200, np.nan, 2500],
         "狀態": ["已出貨", "處理中", "已出貨", "已出貨", "已出貨"] }
df = pd.DataFrame(data)
```

任務二：時間趨勢分析

請讀取下方的 sales_data，並完成：

1. 將「日期」轉為 Datetime 格式。
2. 新增一欄「月份」。
3. 統計並印出「各月份」的營收總和。

```
sales_data = { "日期": ["2026-01-10", "2026-01-15", "2026-02-05", "2026-03-12"],
               "營收": [5000, 7000, 4500, 8800] }
df_sales = pd.DataFrame(sales_data)
```